Building an NLP Paper QA & Knowledge Retrieval System

with LlamaIndex RAG Pipeline

CS6493 Natural Language Processing – Project 3 (Topic 3)

City University of Hong Kong | Q2 2025–2026 | April 2026

Team: TheDayAfterTomorrow

Presentation Outline

01 **Background & Motivation**
Why RAG for NLP paper QA?

02 **System Architecture**
End-to-end pipeline design

03 **Implementation Details**
Key configuration & engineering

04 **QA Test Results**
6 test cases, iterative improvement

05 **LLM Backend Comparison**
Mistral-7B vs. Llama3-8B

06 **Chunking Strategy Analysis**
Memory–performance tradeoffs

07 **Conclusion & Future Work**
Summary and next steps

Team Contributions

Development Team

LEI Jiacheng — Handles Q&A

Core RAG pipeline development, environment setup, dependency resolution

HE Muheng — Presenter

PDF data processing, data source integration via SimpleDirectoryReader

WANG Qingyi

LLM backend integration (Mistral / Llama3), QA test case design & execution

XIA Junxia

Code review, engineering issue tracking, technical material packaging

Report & Presentation Team

ZHU Penglan — Presenter

Report Sections 1–2 (Introduction, Related Work, Architecture), presentation slides

HAN Jinru — Handles Q&A

Report Sections 3–4 (Methodology, Experiments), screenshot integration

ZHUANG Yi

Report Sections 5–6 (Discussion, Conclusion), final formatting & editing

All 7 members contributed to the project • Development: Cells 1–8 in Jupyter Notebook • Report: 6-page final report + slides

Background & Motivation

The Problem

LLMs generate fluent but often unfounded answers when queried about specific research papers

Key limitations:

- Hallucination on domain-specific facts
- No grounding in actual document content
- Cannot reference specific papers or findings

Our Solution: RAG

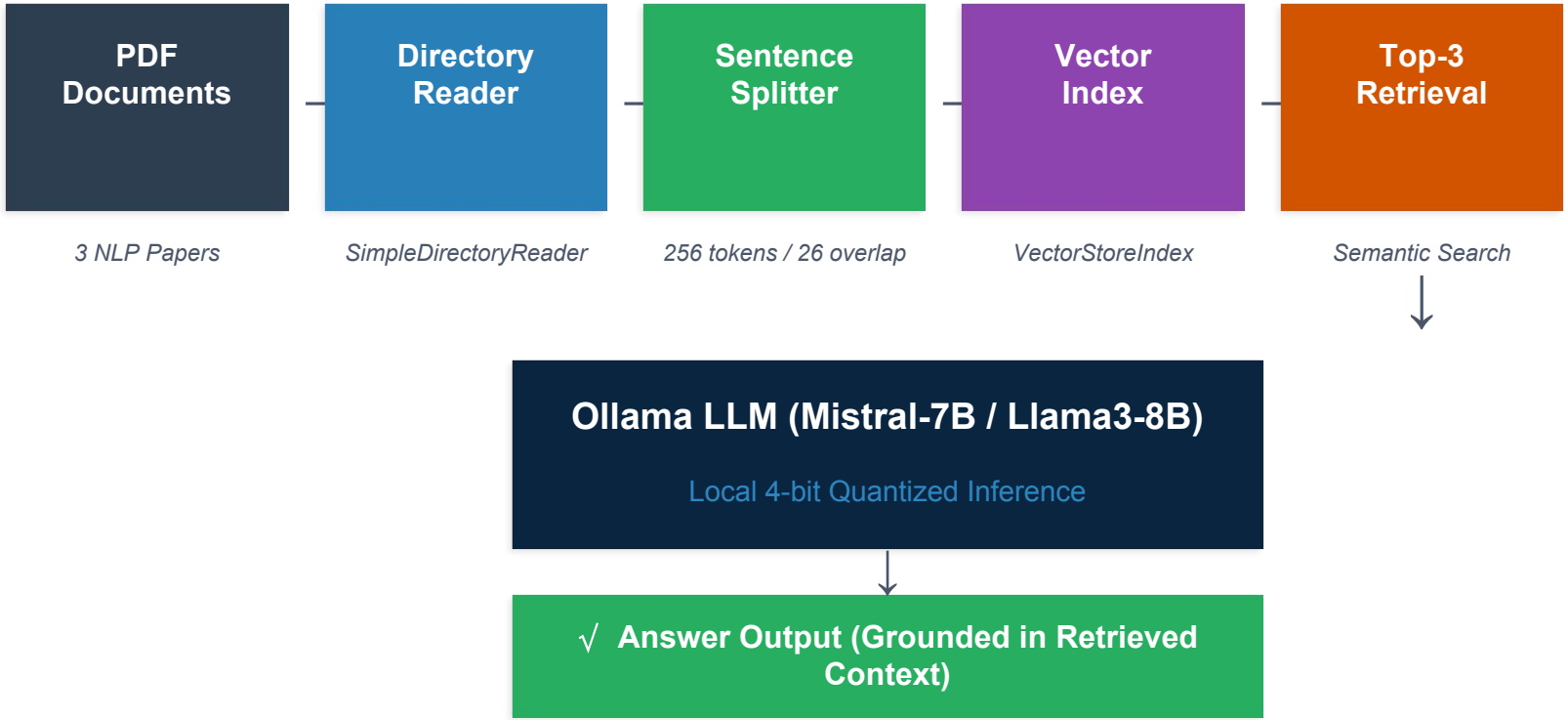
Retrieve relevant chunks from actual papers before generating answers

Key advantages:

- Grounded in real document content
- Reduced hallucination via evidence
- Fully local, no GPU, no API key

3 NLP Papers • 3,312,867 Characters • 17,101 Chunks • 100% Local Execution

System Architecture



Implementation Details

Chunking
256 / 26
tokens / overlap (~10%)

Retrieval
Top-3
Vector similarity search

Embedding
Mock (384d)
Offline fallback mode

LLM
Mistral-7B
4-bit via Ollama

6 Engineering Issues Resolved

- Dependency conflicts → Silent install without version pins
- HuggingFace timeout → 4 offline env variables + MockEmbedding fallback
- Deprecated API (RetrieverQueryEngine) → Switched to index.as_query_engine()
- Low test hit rate (67%) → Redesigned questions anchored to PDF keywords (100%)

QA Test Results

#	Test Question	Source	Result
Q1	Sentiment analysis in NLP research?	Paper 1&3	√ Correct
Q2	LDA and topic modeling in documents?	Paper 3	√ Correct
Q3	ChatGPT in financial NLP tasks?	Paper 1	√ Good
Q4	Challenges of IE and NER in NLP?	Paper 2&3	√ Correct
Q5	Word embeddings and their role?	All 3	√ Correct
Q6	Topic modeling algorithms (LDA)?	Paper 3	√ Correct

Iterative Improvement

Metric	Initial	Redesigned	Change
Task Completion	66.7%	100%	**+33.3%**
Retrieval Hit Rate	67%	100%	**+33%**
Hallucination	17%	0%	**Eliminated**

LLM Backend Comparison

Mistral-7B-q4

6.13s

response time

- 4-bit quantized, low memory
- Concise, application-focused answers
- **Best for RAG real-time QA**

Llama3-8B

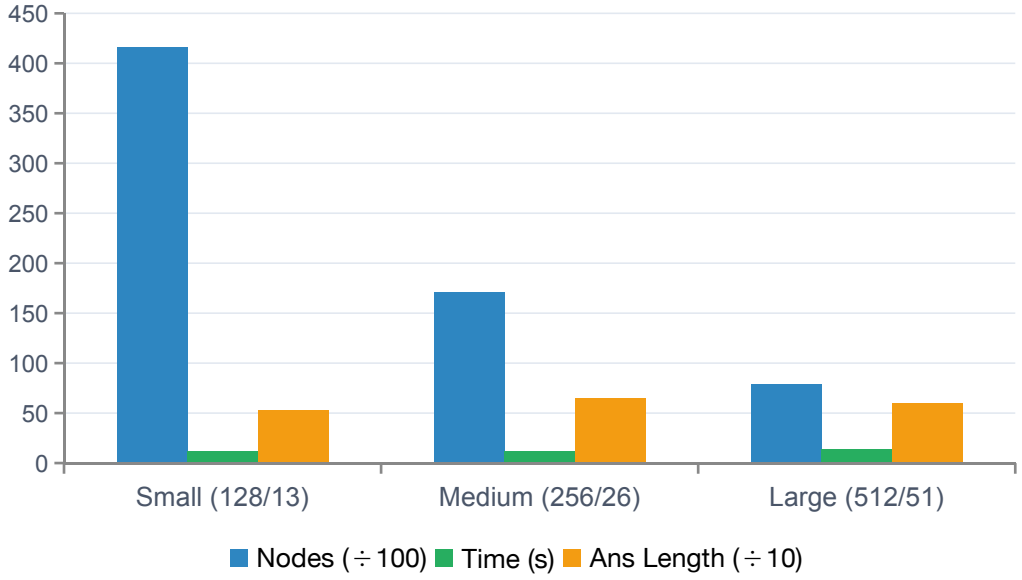
17.18s

response time

- Default precision, higher memory
- Academic, definition-rigorous style
- **Better for in-depth academic QA**

Verdict: Mistral-7B-q4 is 2.8× faster with comparable accuracy — optimal for this project

Chunking Strategy Analysis



Key Findings

- 256/26 produces the longest, most complete answers (651 chars)
- Response time similar across all three strategies (11–14s)
- Memory anomaly at 256 due to MockEmbedding (not real-world behavior)
- LLM inference is the primary latency bottleneck, not retrieval

256/26 = Best Balance

Highest answer quality (651 chars) + Fastest response (11.15s) — validates course parameter selection

Discussion & Limitations

MockEmbedding

Random vectors, no true semantic similarity. Retrieval success depends on keyword density, not meaning. Fragile for arbitrary queries.

Test Case Design

Questions were engineered to match high-frequency terms. This ensures demo success but doesn't represent real-world query diversity.

Single Domain

Only NLP papers tested. Cross-domain generalization untested. The system may need tuning for different academic fields.

Despite limitations, the system achieves 100% task completion and validates the full RAG pipeline

Conclusion & Future Work

100%	2.8×	651	6
Task Completion (6/6 QA Tests)	Speed Advantage (Mistral vs Llama3)	Best Answer Length (256/26 Chunking)	Issues Resolved (Engineering Fixes)

Future Improvements

- Replace MockEmbedding with all-MiniLM-L6-v2 for true semantic retrieval
- Experiment with larger chunk sizes (512) to reduce context truncation
- Extend data connectors to support web pages and plain text files
- Implement human feedback mechanisms for answer quality assessment

Thank You • Q&A